

Лабораторная работа №9.

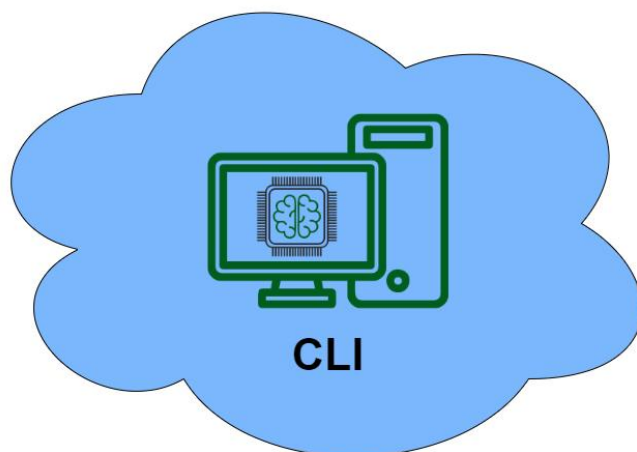
Работа с процессами и заданиями

Цель работы:

Ознакомиться с управлением процессами и заданиями в операционной системе ALT Linux, научиться взаимодействовать с процессами с помощью командной строки, отслеживать состояние процессов и управлять заданиями. Понять основы многозадачности и приоритетов процессов в UNIX-подобных системах.

Оборудование, ПО:

- ALT1
- Справочная литература или доступ в сеть интернет.



Машина	IP	Маска	Шлюз	DNS
CLI (Альт Рабочая станция 10.2)	192.168.1.1	24	-	77.88.8.8

Порядок работы:

1. Переименовать ПК своей фамилией
2. Вставить скриншоты результатов работы каждой команды
3. Запустить на ПК различные приложения, браузеры, и приступить к выполнению контрольных заданий.

Контрольные вопросы:

1. Что такое процесс в операционной системе? Какие основные состояния процесса существуют?
2. Что такое приоритет процесса? Как изменить приоритет уже запущенного процесса?

3. Что такое zombie-процессы, и как они возникают? Как их устранить?
4. Что такое фоновые и приостановленные процессы? Как с ними взаимодействовать с использованием команд оболочки?

Контрольные задания:

1. Используя команду ps, отобразите все процессы, запущенные текущим пользователем и отдельно пользователем root. Найдите процессы, которые работают в фоновом режиме.
2. С помощью команды top или htop отследите потребление ресурсов текущими процессами (CPU, память) и выведите топ-5 процессов, потребляющих наибольшее количество CPU.
3. Запустите долгий процесс (создайте для этого скрипт, который будет бесконечно выполняться) и переведите его в фоновый режим. Остановите выполнение процесса, а затем возобновите его выполнение с помощью “команд управления заданиями”.
4. Измените приоритет процесса так, чтобы он был стал самым значимым в системе. Проанализируйте количество выделяемых на него ресурсов до изменения и после.
5. С помощью утилиты htop измените приоритет выполнения любого процесса на самый минимальный и максимальный.
6. Сконфигурируйте меню htop, добавив колонки, которые будут показывать ID родителя процесса и терминал, на котором выполняется процесс.
7. В htop проанализируйте с помощью “поиска” или “фильтрации”, какие процессы запускаются при открытии браузера Firefox. Что произойдет, если начать писать или выполнять поиск в браузере? На какие процессы и библиотеки системы выделяет большее количество ресурсов?
8. С помощью утилиты htop отправьте сигнал остановки запущенному “бесконечному” скрипту.
9. Найдите в вашей системе процессы-зомби. Почему вы смогли/не смогли их найти?

Используемые источники:

- Официальная документация операционной системы Альт Рабочая Станция 10.4 <https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html/alt-workstation/index.html>

Выполняющаяся что-либо в системе программа называется процессом. Каждый процесс имеет свое собственное пространство памяти, и может быть запущен в определенном состоянии (например, готов к выполнению, в ожидании, заблокирован и т. д.). Таким образом, процесс включает в себя информацию о состоянии выполнения программы (например, инструкции, регистры процессора, стек вызовов и переменные) и ресурсы, необходимые для работы (например, память, файлы, устройства ввода-вывода). Операционная система управляет процессами, назначая им ресурсы, устанавливая приоритеты выполнения, планируя их работу и обеспечивая безопасность выполнения.

Все процессы система регистрирует в таблице процессов, присваивая каждому уникальный номер — идентификатор процесса (process identifier, PID).

Чтобы получить список всех процессов в расширенном формате, необходимо в терминале ввести команду:

```
$ ps -ef
```

Результат работы

Каждая строка вывода представляет один процесс и содержит информацию о его идентификаторе (PID), пользователе, использовании CPU, памяти и **других характеристиках**.

Укажите расшифровку каждой характеристики процесса

Каждая следующая запущенная программа имеет PID на единицу больше. Таким образом PID пользовательских программ доходит уже до нескольких тысяч. Файл `/proc/sys/kernel/pid_max` содержит максимально возможное значение для PID. При достижении этого значения будет достигнут предел и создание новых процессов станет невозможным.

Листинг файла

По умолчанию значение `/proc/sys/kernel/pid_max` может быть установлено на 4194304. Увеличение этого значения может быть полезным в системах с большим количеством одновременно запущенных процессов.

Состояния процессов

Процесс либо обрабатывается процессором, либо не обрабатывается. Только один процесс может обрабатываться на одном ядре процессора. Все остальные процессы должны ждать или находиться в каком-то другом состоянии.

Список состояний, в которых может находиться процесс:

- R (running — запущенный). Процесс находится на обработке у процессора.
- R (runnable — готов к работе). Процесс готов к обработке и находится в очереди к процессору. Running и runnable обозначаются одинаковой буквой R.
- D (uninterruptible sleep — в беспробудном сне). Когда процесс обращается к устройству, например, к диску или сетевой карте, он переходит в это состояние. А возвращается из него только после получения запрошенной информации. Обычно в таком состоянии процесс находится не долго. Однако процесс перестает обрабатывать любые сигналы и, если что-то пойдет не так, то завершить такой зависший процесс, не перезагружая сервер, не получится.
- I (Idle — бездействующий поток ядра). Состояние похоже на D, но такие процессы не нагружают процессор и исключены из расчета средней нагрузки системы (load average).
- S (sleeping — спит). Процесс ожидает какие-то ресурсы, которые в данный момент недоступны. При переходе в это состояние, процесс немедленно отказывается от доступа к процессору. Когда ресурс, который ожидает процесс, становится доступным, процесс может снова перейти в состояние R (runnable). В спящем состоянии процесс продолжает обрабатывать сигналы.
- T (stopped by job control signal — остановленный специальным сигналом). Про сигналы поговорим позже, тут главное понять, что процессы могут обрабатывать сигналы и один из них может остановить процесс. Это состояние похоже на паузу, то есть из этого состояния процесс может выйти и продолжить свою работу в состоянии R или S.
- Z (zombie — зомби). Когда процесс завершает свою работу, он освобождает свои ресурсы, но не освобождает свой PID в таблице процессов. Вместо этого он отправляет сигнал родителю, сообщая что он завершается. Родительский процесс должен освободить PID дочернего процесса. Отрезок времени между завершением процесса и когда родитель освободит PID завершенного дочернего процесса называется состоянием зомби. Процесс может остаться в состоянии Zombie, если родительский процесс умрет до того, как освободит PID дочернего процесса.

Таблица состояний:

STAT	Описание
R	Выполняется
S	Готов - ожидает выполнения (спит)
D	Ожидает - приостановлен (ввод-вывод)
T	Остановлен (принудительно)
I	Поток ядра
Z	Зомби - завершился, но не был удален
<	высокоприоритетный процесс
N	низкоприоритетный процесс
s	лидер сеанса
L	часть страниц блокирована в памяти
l	мультипоточный (нити)
+	работает в группе переднего плана

Приоритетные/фоновые процессы

Процессы могут быть приоритетными и фоновыми. Приоритетный процесс у нас по сути "прикреплен" к текущему терминалу. То есть он может считывать и передавать данные на стандартные устройства ввода/вывода. Но при этом, терминал будет заблокирован до тех пор, пока запущенный процесс не выполнится. По умолчанию, когда пользователь вводит команду на терминале, создается приоритетный процесс. Это удобно при работе с консольными командами, не требующими значительного времени на выполнение. Но это очень неудобно при использовании различных скриптов, работа которых занимает значительный период времени.

Для таких случаев лучше использовать фоновые процессы. Такие процессы не занимают текущий терминал, то есть мы можем одновременно с выполнением фонового процесса работать с другими командами, запускать процессы и т.д. Фоновый процесс также называется **Заданием (Job)**.

Запустить процесс в фоновом режиме очень просто, достаточно **добавить знак &** после команды.

После запуска выводится строка с номером задания (в скобках) и PID, появится приглашение на ввод команды в текущем терминале. При этом, номера заданий относятся только к текущему терминалу.

Приоритетные задачи можно также превращать в фоновые и возвращать обратно в приоритетные.

Пример:

```
$ ping ya.ru
```

Результат работы

Нажатие сочетания клавиш Ctrl-Z приведет к остановке данного процесса.

Результат работы

Далее остановленный процесс можно снова запустить в фоновом режиме с помощью команды **bg [команда]**.

Результат работы

Чтобы запустить процесс в фоновый режим воспользуемся командой **fg [команда]**.

Результат работы

Команды `fg` и `bg` переводят из/в фоновый режим задание с указанным номером. Если номер не указан, действие применяется к последнему созданному заданию.

Иерархия процессов в системе

Процесс `init` традиционно является первым процессом, запускаемым в Unix-подобных системах, который является предком всех других процессов.

В ОС Альт `init` был заменен на `systemd`, который теперь является системным и сервисным менеджером и начальным процессом. Некоторые функции, реализуемые `systemd`, используются для управления системой, например, для монтирования файловых систем, управления устройствами, а также запуска и управления системными службами для запуска системы.

Утилита `ps tree` отображает дерево запущенных процессов, показывая их иерархическую связь. При вызове утилиты `ps tree` без каких-либо параметров, она отобразит дерево процессов всей системы. Верхний / корневой элемент в дереве является родительским процессом для всех системных процессов.

\$ `ps tree`

Результат работы

Утилита объединяет идентичные ветви, помещая их в квадратные скобки и ставя перед ними целое число, представляющее количество ветвей. Это делает вывод более читаемым и визуально привлекательным.

Чтобы отключить слияние идентичных веток, воспользуемся опцией `-c`:

\$ `ps tree -c`

Результат работы

Потоки процесса показаны под родительским процессом и отображаются с использованием имени процесса в фигурных скобках.

Чтобы показать полные имена потоков, воспользуемся опцией `-t`:

```
$ pstree -t
```

Результат работы

Если необходимо скрыть потоки и показать только процессы, используют опцию `-T`:

```
$pstree -T
```

Результат работы

Обычно `pstree` отображает много строк вывода, которые не помещаются на экране. Чтобы просмотреть вывод по одной странице за раз, передайте его команде `less`.

Если в качестве аргумента `pstree` имя пользователя, `pstree` показывает только процессы, принадлежащие этому пользователю.

```
$ pstree user
```

Результат работы

Мониторинг процессов

ps

Манипулируя процессами, система имеет дело именно с их идентификаторами, другого способа отличить один процесс от другого нет. Для просмотра своих процессов можно воспользоваться утилитой ps («process status»):

```
$ ps -f
```

Результат работы

Здесь вызван ps с ключом «-f» («full»), чтобы получить побольше информации.

Команда ps aux показывает все процессы в системе вместе с дополнительной информацией, такой как CPU и память, которую они используют.

Укажите расшифровку каждой характеристике процесса
PID/PPID UID/USER CMD TTY STAT %CPU(C) %MEM VSZ(VIRT) RSS(RES) STIME TIME

top

```
top - 08:56:44 up 1:22, 2 users, load average: 0,21, 0,08, 0,02
Tasks: 247 total, 1 running, 246 sleeping, 0 stopped, 0 zombie
%Cpu(s): 3,1 us, 3,1 sy, 0,0 ni, 93,8 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
MiB Mem : 1967,3 total, 96,0 free, 642,4 used, 1228,8 buff/cache
MiB Swap: 1966,0 total, 1940,6 free, 25,4 used. 1084,6 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	99788	12436	8252	S	0,0	0,6	0:00.99	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.01	kthreadd
3	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	rcu_par_gp
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	netns
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/0:0H-events_highpri
9	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	mm_percpu_wq
10	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_kthread
11	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_rude_kthread
12	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_trace_kthread
13	root	20	0	0	0	0	S	0,0	0,0	0:00.09	ksoftirqd/0
14	root	20	0	0	0	0	I	0,0	0,0	0:00.76	rcu_preempt
15	root	rt	0	0	0	0	S	0,0	0,0	0:00.03	migration/0
16	root	20	0	0	0	0	I	0,0	0,0	0:00.88	kworker/0:1-mm_percpu_wq
17	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/0
18	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/1
19	root	rt	0	0	0	0	S	0,0	0,0	0:00.23	migration/1
20	root	20	0	0	0	0	S	0,0	0,0	0:00.12	ksoftirqd/1
22	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/1:0H-events_highpri
23	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kdevtmpfs
24	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	inet_frag_wq
25	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kauditd
26	root	20	0	0	0	0	S	0,0	0,0	0:00.00	khungtaskd
28	root	20	0	0	0	0	S	0,0	0,0	0:00.00	oom_reaper

Интерактивная утилита top отображает список процессов, потребляющих больше всего системных ресурсов (CPU и память), обновляя информацию в реальном времени. Также показывает общую статистику системы. Установлена по умолчанию.

Инструмент top позволяет сортировать список процессов:

Клавиша М – для сортировки по памяти;

Клавиша Р – для сортировки по использованию CPU;

Клавиша N – для сортировки по идентификатору процесса;

Клавиша Т – для сортировки по времени работы.

По умолчанию top не показывает полный путь к программе и не делает различие между процессами ядра и процессами пользовательского пространства. Если вам нужна эта информация, нажмите **c**. Нажмите c еще раз, чтобы вернуться к отображению по умолчанию.

Результат работы

--

Иногда в процессе работы может понадобиться иерархия процессов. Вы можете раскрыть ее с помощью клавиш **V** во время работы **top**.

Результат работы

Чтобы вывести процессы определенного пользователя, нажмите **u**, затем введите имя пользователя или оставьте его пустым, чтобы отобразить процессы для всех пользователей.

Выведите все процессы пользователя root

Вы можете нажать **t** и **m**, чтобы изменить стиль статистики процессора и памяти.

Результат работы

Интерфейс можно разделить на две рабочие зоны:

Верхняя рабочая зона содержит сведения о времени работы сервера, свободных и занятых ресурсах, пользователях.

Основная рабочая зона – динамически обновляемая таблица, содержащая сведения о процессах.

На первой строке показаны следующие поля:

- текущее время;
- время безотказной работы системы;
- количество активных сеансов пользователей;

- средняя загрузка системы (load average) — три величины, усредненные за последние 1, 5 и 15 минут. Чем ниже, тем лучше. Если превышает количество ядер на сервере, то значит сервер перегружен.

Результат работы

В разделе Tasks отображается статистика процессов, выполняемых в вашей системе. Далее перечислено общее количество процессов, активные, спящие, остановленные и процессы-зомби.

Раздел использования CPU показывает процентное время процессорного времени, затрачиваемого на различные задачи:

- us — потребление CPU пользовательскими процессами (top);
- sy — потребление CPU системными процессами;
- ni — потребление CPU пользовательскими процессами, с измененным приоритетом (командой nice или renice);
- id — простаивание CPU;
- wa — процент времени, в течение которого процессор простаивал, ожидая завершения операции ввода-вывода. Если слишком высокое (более 10), значит за вашим процессором не поспевает диск, и нужно бы поменять его на более быстрый;
- hi — процент времени работы аппаратных прерываний. Периферийные устройства могут прерывать работу процессора, чтобы передать какие-то данные;
- si — процент времени работы программных прерываний. Некоторые приложения могут прерывать работу процессора, чтобы передать какие-то данные;
- st — процент времени, когда реальный процессор не был доступен виртуальной машине. Будет расти если виртуальной машине выделить больше ядер, чем имеет гипервизор. Этот показатель имеет значение только в системах виртуализации.

Проведите анализ нагрузки на вашей системе. Интерпретируйте каждый параметр.
--

Последние 2 строки показывают информацию об использовании памяти в системе. Строки Mem и Swap отображают информацию о RAM и области подкачки соответственно.

total — общее количество памяти;

free — количество свободной памяти;

used — количество используемой памяти;

buff / cache — память, выделенная под буфер и дисковый кэш;

avail Mem — память которая может быть выделена для запуска новых процессов.

htop

```
0[|||||] 15.0% Tasks: 174, 125 thr, 73 kthr; 1 running
1[|||] 4.8% Load average: 0.25 0.08 0.02
Mem[|||||] 638M/1.92G Uptime: 01:22:08
Swp[|] 19.5M/1.92G

Main I/O
PID TTY PPID TIME+ USER PRI NI VIRT RES SHR S CPU%MEM% TIME+ Command
2594 tty1 2576 1:36.11 root 20 0 497M 144M 69696 S 4.1 7.4 1:36.11 X -nolisten tcp :0 -seat seat0 -auth /run/light
2732 tty1 2576 0:00.33 root 20 0 497M 144M 69696 S 2.0 7.4 0:00.33 X -nolisten tcp :0 -seat seat0 -auth /run/light
3308 (no tty) 1 0:01.22 user 20 0 829M 58220 44208 S 2.0 2.9 0:01.22 mate-terminal
4862 pts/0 3541 0:00.05 root 20 0 5820 4044 3040 R 1.4 0.2 0:00.05 htop
2589 (no tty) 2567 0:00.29 zabbix 20 0 15908 5536 4344 S 0.7 0.3 0:00.29 /usr/sbin/zabbix_agentd: listener #1 [waiting f
3084 (no tty) 2950 0:00.15 user 20 0 259M 20260 14184 S 0.7 1.0 0:00.15 /usr/bin/mate-session
3270 (no tty) 3084 0:13.07 user 20 0 420M 32176 20732 S 0.7 1.6 0:13.07 marco
3290 (no tty) 3084 0:00.85 user 20 0 1228M 66060 53028 S 0.7 3.3 0:00.85 /usr/bin/caja
1 (no tty) 0 0:00.99 root 20 0 99788 12436 8252 S 0.0 0.6 0:00.99 /sbin/init splash
2234 (no tty) 1 0:00.25 root 20 0 46416 18700 15560 S 0.0 0.9 0:00.25 /lib/systemd/systemd-journald
2267 (no tty) 1 0:00.24 root 20 0 25996 6616 3856 S 0.0 0.3 0:00.24 /lib/systemd/systemd-udevd
2387 (no tty) 1 0:00.11 root 20 0 230M 9104 6220 S 0.0 0.5 0:00.11 /usr/libexec/accounts-daemon
2389 (no tty) 1 0:00.14 root 20 0 50568 14532 6672 S 0.0 0.7 0:00.14 /usr/bin/guile --no-auto-compile /usr/sbin/alte
2390 (no tty) 1 0:00.04 _avahi 20 0 7784 4436 3856 S 0.0 0.2 0:00.04 avahi-daemon: running [alt.local]
2393 (no tty) 1 0:00.53 messagebus 20 0 9524 5992 4064 S 0.0 0.3 0:00.53 /bin/dbus-daemon --system --address=systemd: --
2396 (no tty) 1 0:00.05 root 20 0 230M 9104 6220 S 0.0 0.5 0:00.05 /usr/libexec/accounts-daemon
2400 (no tty) 1 0:00.23 polkitd 20 0 2529M 21588 14532 S 0.0 1.1 0:00.23 /usr/libexec/polkit-1/polkitd --no-debug
2404 (no tty) 1 0:00.14 root 20 0 22056 7444 6540 S 0.0 0.4 0:00.14 /lib/systemd/systemd-logind
2405 (no tty) 1 0:00.10 root 20 0 387M 15356 10296 S 0.0 0.8 0:00.10 /usr/libexec/udisks2/udisksd
2406 (no tty) 1 0:00.06 root 20 0 227M 8420 5800 S 0.0 0.4 0:00.06 /usr/libexec/upower/upowerd
2416 (no tty) 2390 0:00.00 _avahi 20 0 7784 404 0 S 0.0 0.0 0:00.00 avahi-daemon: chroot helper
2423 (no tty) 1 0:00.00 root 20 0 387M 15356 10296 S 0.0 0.8 0:00.00 /usr/libexec/udisks2/udisksd
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8NICE F9Kill F10Quit
```

Утилита htop является улучшенной версией top, предоставляющей более удобный пользовательский интерфейс, возможности для взаимодействия с процессами (например, завершение процессов) и более гибкие настройки отображения.

Управление:

F1 — справка;

F2 — настройки;

F3 — поиск процесса;

F4 — сортировка списка процессов;

F5 — устанавливает древовидное отображение;

F6 — открывает панель с выбором параметра сортировки процессов;

F7 — увеличивает приоритет выполнения текущего процесса;

F8 — уменьшает приоритет выполнения текущего процесса;

F9 — передать сигнал процессу;

F10 — выходит из программы.

Кнопка Home перенесёт к началу списка.

Кнопка End прокрутит список до конца.

Ctrl-a или ^ — прокрутка к началу пункта (началу строки).

Ctrl-e или \$ — прокрутка к концу пункта записи о процессе (то есть концу строки).

Приоритеты выполнения процессов

Niceness (уступчивость)

Представляет собой значение приоритета, которое процессы могут устанавливать, чтобы уступать CPU другим процессам. Значение `niceness` варьируется от -20 (высокий приоритет) до 19 (низкий приоритет). Утилиты `top` и `htop` позволяют изменять значение `niceness` для процессов в реальном времени, что может быть полезно для управления нагрузкой на систему.

NICE может быть в пределах от -20 до 19. По умолчанию 0.

Утилита `nice` используется для задания приоритета процесса.

Пример (в терминале):

```
$ nice -n 15 ping ya.ru > /dev/null &
```

Результат работы

```
$ ps ax -o pid,ppid,cmd,nice,etime
```

Результат работы

Утилита `renice` используется для изменения приоритета процесса.

Пример (в терминале):

```
$ renice 10 3833
```

Результат работы

```
$ renice 10 $(pgrep firefox)
```

Результат работы

Сигналы

В Linux сигналы используются для управления процессами. Сигналы – это асинхронные уведомления, которые могут отправляться одному процессу (процессу-получателю) от другого процесса (процесса-отправителя) или от операционной системы.

Сигналы могут быть:

- Обработываемые самим процессом (игнорируемые)

Процессы могут быть запрограммированы на обработку определённых сигналов специфическим образом, включая их игнорирование.

Некоторые сигналы, такие как SIGKILL и SIGSTOP, не могут быть перехвачены или обработаны процессами и всегда обрабатываются ядром.

- Обработываемые ядром

Процессы могут установить обработчики сигналов, чтобы определить поведение при получении конкретного сигнала, например, процесс может обработать сигнал SIGTERM для корректного завершения работы. Кроме того, сигналы могут использоваться для синхронизации процессов, уведомления о событиях или для обеспечения безопасного завершения процессов.

Чтобы завершить работу фонового процесса с помощью `<<^C>>`, необходимо сначала сделать его активным. Это не всегда возможно, и не всегда удобно. Сигнал — это способность процессов обмениваться стандартными короткими сообщениями непосредственно с помощью системы. Сообщение-сигнал не содержит никакой информации, кроме номера сигнала (для удобства вместо номера можно использовать предопределённое системой имя). Для того, чтобы передать сигнал, процессу достаточно задействовать системный вызов **kill()**, а для того, чтобы принять сигнал, не нужно ничего. Если процессу нужно как-то по-особенному реагировать на сигнал, он может зарегистрировать обработчик, а если обработчика нет, за него отреагирует система. Как правило, это приводит к немедленному завершению процесса, получившего сигнал. Обработчик сигнала запускается асинхронно, немедленно после получения сигнала, чтобы процесс в это время ни делал. Сигнал — короткое сообщение, посылаемое системой или процессом другому процессу. Обработывается асинхронно специальной подпрограммой-обработчиком. Если процесс не обрабатывает сигнал самостоятельно, это делает система.

Два сигнала — 9 (KILL) и 19 (STOP) — всегда обрабатывает система. Первый из них нужен для того, чтобы убить процесс наверняка (отсюда и название). Сигнал STOP приостанавливает процесс: в таком состоянии процесс не удаляется из таблицы процессов, но и не выполняется до тех пор, пока не получит сигнал 18 (CONT) — после чего продолжит работу. В Linux сигнал STOP можно передать активному процессу с помощью управляющего символа `<<^Z>>`:

Посылка сигналов:

```
$ kill [ -SIGNAL ] PID
```

Приведите 2 примера “убийства” процесса браузера Firefox в системе по номеру сигнала и названию. Использовать сигнал SIGKILL. Листинг команд приведите ниже.


```
$ killall [ -SIGNAL ] { user | name }
```

Запустите множество утилит на ваш выбор. Сначала “остановите” все процессы текущего пользователя, а затем их возобновите. Листинг команд приведите ниже.

```
$ pkill [options] pattern
```

Запустите браузер Firefox и почтовый сервис ThunderBird. “Убейте” их. Листинг команд приведите ниже.

Часто используемые сигналы:

№	Имя	Описание
1	HUP	обрыв связи,
2	INT	Прервать (Ctrl+C)
9	KILL	Уничтожить
15	TERM	Завершить (сигнал по умолчанию)
18	CONT	Продолжить после STOP
19	STOP	Приостановить
20	TSTP	Стоп с клавиатуры (Ctrl+Z)